

Pullsrc

Paste a page url. Get every media and design asset on it, ready to use and properly credited.

PRD · DRD · System Design · UX Document

v1 scope · single-page url only · zero-infra-cost build

1. Product Requirements Document (PRD)

Problem

Content and design teams regularly need to pull media (images, video, fonts, logos) from a live webpage to reuse in their own work, most often with credit given to the original source. Today this is manual: right-click save, dig through devtools, or give up entirely when a site hides its video behind a chunked streaming player instead of a plain file link. This broke a real workflow: a newsletter team could not download a video from a source page because the video loaded in segments, not a single file.

Goal

Let anyone paste one page url and get back every usable asset on that page, categorized, previewable, downloadable individually or in bulk, with a ready-to-paste source credit line for each item.

Target user

- Newsletter and content writers sourcing media for articles, need speed and correct attribution.
- Designers doing competitor or inspiration research, need fonts, logos, and color palettes fast.
- Indie devs and marketers pulling brand assets or reference material from a live site.

V1 scope, in

- Single page url input only. No domain-wide or multi-page crawling in v1.
- Asset detection: images, fonts, logo (best-guess), icons, GIFs, direct-link video, color palette.
- Streaming video (HLS/DASH) detection and client-side conversion to a downloadable file.
- Auto-generated source credit line per asset, plus a combined credit sheet for the whole page.
- Individual download per asset, and download-all-as-zip.

V1 scope, out (deferred)

- Domain-wide or bare-domain crawling. Killed by design, not a v1 gap, see UX Document section 2.
- 3D model, audio/sound-effect, Lottie animation, PDF/document, favicon-set, and subtitle extraction. Planned as an expandable "more assets found" section post-v1, not core tabs.
- DRM-protected streaming video. Not solvable by design, will be labeled clearly, not silently failed.
- Team accounts, shared libraries, browser extension, batch multi-url scanning.

Success signal for v1

A real user can paste a page that previously failed to yield a downloadable video, and walk away with a correctly stitched file plus a copy-pasteable credit line, without touching devtools.

2. UX Document

2.1 The core UX problem: url vs domain

Early design considered accepting a bare domain (e.g. amazon.com) and asking the user how many pages to crawl, or a fixed random sample. Both are wrong: the user cannot know which pages they are getting, so the output cannot be trusted. The actual triggering use case was always a single article page, not a whole site.

Decision: v1 accepts a specific page url only. No page-count picker, no random crawl.

If the input looks like a bare domain (no path, e.g. just "amazon.com"), the app does not attempt to guess scope. It shows an inline message asking the user to paste the specific page they mean, with one example shown inline (e.g. "amazon.com/product-page"). This keeps trust intact: what the user pastes is exactly what gets scanned, nothing more.

Deferred, correct way to eventually support more than one page: let the user paste a specific list of urls, or show them the site's actual page list (from a sitemap) by real title so they tick named pages, never a blind page-count or random sample.

2.2 Core flow

- **1. Input.** One large field on landing. User pastes a page url. Auto-detect bare-domain vs page and branch per 2.1.
- **2. Scanning state.** Live-feeling progress, asset categories appear as they are found with counts ticking up (Images, Fonts, Logo, Icons, Colors, Video). Not a plain spinner.
- **3. Streaming-video state (conditional).** If HLS/DASH detected, a distinct sub-state: "Converting streaming video" with its own progress, since this takes longer than everything else and runs client-side.
- **4. Results.** Tabbed by category (All / Images / Fonts / Logo / Icons / Colors / Video). Each asset: thumbnail, file type, size, source tag, inline credit line with a copy button.
- **5. Bulk actions.** Multi-select checkboxes, Download Selected, Download All as Zip.
- **6. Credit sheet.** One clean page/tab listing every asset's full attribution text as one copyable block, for pasting under an article.

2.3 Required states, not optional

- Zero assets found on the page.
- Broken or unreachable url.
- Page blocked or paywalled (scan cannot render it).
- Streaming video detected but DRM-protected, labeled clearly, not a silent failure.
- Logo detection shown as "likely logo" with a thumbs up/down, never asserted as certain.

2.4 Visual design direction

- Dark theme by default. Builder/dev-tool aesthetic, closer to Linear or Vercel dashboard than generic SaaS.
- One distinct small icon per asset category, used consistently across scanning state, tabs, and results.
- Generous whitespace. Never show every category crammed into one dense view.
- Motion is purposeful only: the scan/populate moment gets subtle animation, nothing else does.
- Zero jargon in the UI. A first-time user should need no explanation.

3. System Design

3.1 Constraint driving this architecture

This is a side project with no budget for dedicated infrastructure. Every architectural choice below optimizes for zero or near-zero standing cost, while still solving the hard technical problem (chunked/streaming video) properly rather than skipping it.

3.2 Component overview

- **Frontend (Next.js on Vercel).** Input handling, scanning UI, results grid, credit sheet, all states from the UX document.
- **Scan API route (Vercel serverless function).** Runs a headless browser to render the target page fully (catches lazy-loaded, JS-injected, and CSS-background assets that a plain HTML fetch would miss) and listens to actual network responses rather than reading the DOM, since players hide real video urls behind blob urls in the DOM.
- **Client-side video conversion (ffmpeg.wasm, runs in the user's browser).** When a streaming manifest (.m3u8 / .mpd) is detected, the browser fetches the segments and stitches them into one mp4 locally. No server-side video processing, no dedicated compute bill.
- **Thin proxy route (Vercel serverless function).** Some source sites block cross-origin segment fetching (CORS). This route relays segment bytes only, no processing, keeping the muxing itself client-side.
- **Credit metadata step.** Runs alongside the scan, attaches source domain, page title, original asset url, and scan date to every found asset, used to render the per-asset and combined credit text.

3.3 Data flow

- User submits a page url.
- Scan function launches headless Chromium (puppeteer-core + a Vercel-compatible Chromium build), loads the page, and records every qualifying network response by type: images, fonts, direct video files, and streaming manifests/segments.
- Results, categorized and with credit metadata attached, are returned to the frontend as soon as the scan completes.
- If a streaming manifest was found, the frontend hands its url to ffmpeg.wasm in the browser, which fetches segments (via the proxy route if needed) and produces one mp4 file client-side.
- User downloads individual assets directly, or triggers a client-built zip containing every category folder plus a generated credit-sheet.txt.

3.4 Output structure (zip download)

```
pullsrc-export/  
images/  
fonts/  
logo/
```

```
icons/  
video/  
credit-sheet.txt
```

3.5 Known limitations, stated up front

- DRM-protected streams cannot be extracted. This is intentional by the platform being scraped, not a gap in this build. The UI must label this clearly rather than fail silently.
- Vercel serverless functions have execution time limits, fine for a single-page scan, which is why video muxing was deliberately moved to the client rather than attempted server-side.
- ffmpeg.wasm is slower than native ffmpeg and runs on the user's device, long videos take noticeably longer and use more browser memory. Acceptable trade for zero infrastructure cost.
- Logo and icon detection are heuristic, not guaranteed correct, and are presented to the user as best-guess.

3.6 Tech stack

Layer	Choice	Why
Frontend + hosting	Next.js on Vercel	Zero-cost tier fits side-project scale, fast to ship
Page rendering / scan	puppeteer-core + @sparticuz/chromium	Runs inside Vercel serverless limits
Video detection	Network-response interception	Real video urls only exist at network layer, not DOM
Video muxing	ffmpeg.wasm, client-side	No server compute cost, avoids a second paid service
CORS handling	Thin Vercel proxy route	Relays bytes only, not a processing service

4. Design Requirements Document (DRD)

4.1 Design principles

- **Trust over cleverness.** The user must always know exactly what was scanned and why. No silent scope expansion, no guessed page counts, no unexplained failures.
- **Honesty about uncertainty.** Anything inferred rather than certain (logo, likely-icon detection) is visually marked as a best guess, never presented as fact.
- **One clear path per screen.** Every state (scanning, results, error, empty) has exactly one obvious next action, never a dead end.

4.2 Layout requirements

- Landing screen: single dominant input, minimal copy, no marketing clutter above the fold.
- Results screen: category tabs across the top or a left rail, asset grid below, bulk-action bar persistent at the bottom or top while items are selected.
- Credit sheet: a distinct, printable-feeling view, optimized for one clean copy action, not styled like the results grid.

4.3 Interaction requirements

- Bare-domain input must never silently proceed, the redirect-to-specific-page message (2.1) is a hard requirement, not a nice-to-have.
- Every asset card shows its credit line inline with its own copy button, the user should never have to visit a separate screen to get one citation.
- The streaming-video conversion state must be visually distinct from the normal scan, since it takes meaningfully longer and runs in-browser.

4.4 Accessibility and clarity baseline

- No unexplained icons without a text label or tooltip on first use.
- Color is never the only signal for state (e.g. error vs success), pair with icon and text.
- Every category tab and action button must be understandable without prior context, this is a tool for someone who has never used it before, every time.